1 Combinator Calculus

1.1 Combinator Algebra

Definition 1.1 (terms, and combinators)**.** Fix two *constants*, K and S, and a stock of *variables* $x, y, z, f, g, \ldots$. The *terms* are defined inductively:

1. Each constant, K and S, is a term.
2. Each variable x is a term.
3. If M and N are terms, then $(M\,N)$ is a term: the *application* of M to N.
4. Nothing else is a term.

A term without any variables is a *combinator*.

In one line, with M and N standing for terms already built,

$$M, N ::= \mathsf{K} \mid \mathsf{S} \mid x \mid (M\,N)$$

Writing out every bracket gets tiresome, so we adopt the convention where **application groups to the left**. Thus, $\mathsf{K}\,x\,y$ means $((\mathsf{K}\,x)y)$ and we only write brackets when the grouping is *not* to the left: $f\,(g\,x)$ and $f\,g\,x$ are different terms.

On their own, terms just sit there. We allow them exactly one type of action, *rewriting*: we write $M \rightsquigarrow N$ to say M *steps to* N, or one rewrite turns M into N.

Definition 1.2 (reduction)**.** The relation $M \rightsquigarrow N$, is given by one of the rules:

1. Rule K has nothing above the bar, so it holds outright. K keeps the first of two arguments and drops the second.

$$\frac{}{((\mathsf{K}\,P)\,Q) \;\rightsquigarrow\; P}\;\mathsf{K}$$

2. Rule S also has nothing above the bar, so it holds outright. S hands its third argument to both of the others and applies the first result to the second.

$$\frac{}{(((\mathsf{S}\,P)\,Q)\,R) \;\rightsquigarrow\; ((P\,R)\,(Q\,R))}\;\mathsf{S}$$

3. Rule (app_1) says that if $P \rightsquigarrow P'$ then $P\,Q \rightsquigarrow P'\,Q$: a step in the function is a step of the application.

$$\frac{P \rightsquigarrow P'}{(P\,Q) \;\rightsquigarrow\; (P'\,Q)}\;\text{app}_1$$

4. Rule (app_2) says the same of the argument.

$$\frac{Q \rightsquigarrow Q'}{(P\,Q) \;\rightsquigarrow\; (P\,Q')}\;\text{app}_2$$

Example 1.4 (the identity)**.** Here is the run of I on an argument:

$$\begin{aligned} \mathsf{S}\,\mathsf{K}\,\mathsf{K}\,x &\rightsquigarrow \mathsf{K}\,x\,(\mathsf{K}\,x) && \text{the } \mathsf{S} \text{ step, with } P = Q = \mathsf{K} \text{ and } R = x \\ &\rightsquigarrow x && \text{the } \mathsf{K} \text{ step: keep } x\text{, drop } \mathsf{K}\,x. \end{aligned}$$

The function that returns its argument unchanged isn't one of the two constants, but the language has it all the same. Whatever x is, $\mathsf{I}\,x \rightsquigarrow^* x$.

Terms built from constants by application, together with rules for rewriting them, make a *combinatory algebra*, and K and S are its basic combinators.

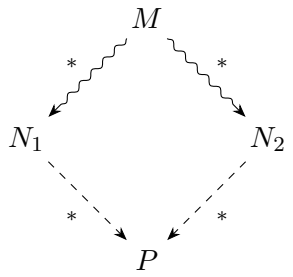
1.2 Confluence

The rules (app_1) and (app_2) let a step happen anywhere inside a term, so a term can often step in more than one way. A **redex**, short for *reducible expression*, is a term with the shape of the left of (K) or (S):

$$\mathsf{K}\,P\,Q \qquad\qquad \mathsf{S}\,P\,Q\,R$$

What the rule puts in its place is the **reduct**. A step rewrites exactly one redex.

Definition 1.6 (Church–Rosser; on loan)**.** If $M \rightsquigarrow^* N_1$ and $M \rightsquigarrow^* N_2$, then there is a term P such that $N_1 \rightsquigarrow^* P$ and $N_2 \rightsquigarrow^* P$. It's usually remembered as a picture. The two runs at the top are given; the theorem promises the two dashed runs at the bottom.



This property is called *confluence*. Some terms can step in more than one way; others can't step at all. A term with no redex is in *normal form*. Machines waiting for inputs, such as $\mathsf{K}\,x$ and $\mathsf{S}\,x\,y$, and K, S and variables on their own, are all in normal form.

1.3 Evaluation Strategies

An implementation of the combinator calculus has to decide which redex to rewrite when a term has more than one. The choice can't change the result, but it can change how much work it takes. Two strategies matter:

1. *Strict, or eager.* Rewrite the **innermost** redex first, one with no redex inside it, and work outwards. In programming terms: run the arguments, before the call.
2. *Lazy.* Rewrite the **outermost** redex first, one that sits inside no other, work inwards. In programming terms, run an argument only when the function needs it.

The default is a bet on which behaviour we want more often:

1. *Predictable cost favours strict.* The work for an argument is paid at the call, so we can read the cost off the line. A lazy sum of a long list can pile up a million unrun additions.
2. *Effects favour strict.* Printing, reading, raising an error happen in the order written. Lazily, an effect happens when its result is demanded.
3. *Modularity favours lazy.* One part of a program describes an enormous list; another takes what it needs. Python's generators borrow the idea.

Everything in this section assumes that the run ends, and confluence says that two runs of a term can be brought back together without saying anything about either stopping.

1.4 Programming

Running is plainly computing: a term goes in, a normal form comes out.

Definition 1.11 (definitions)**.** A *definition* is an equation

$$f\,x_1 \ldots x_n \coloneqq M,$$

where:

- f is a new name;
- $x_1, \ldots, x_n$ are distinct variables;
- M is a term built from constants, names defined earlier, and the variables $x_1, \ldots, x_n$ alone.

Once defined, the name f may be used as a term, and it steps by the rule

$$\frac{}{((\cdots((f\,P_1)\,P_2)\cdots)\,P_n) \;\rightsquigarrow\; M \text{ with } P_i \text{ for each } x_i}\ \text{def}$$

alongside the rules of Definition 1.2.

The rule (def) says that wherever f stands with n arguments, it may be replaced by the right-hand side, with the arguments written in for the parameters; with (app_1) and (app_2), this can happen anywhere in a term. The I above is now official with definition $\mathsf{S}\,\mathsf{K}\,\mathsf{K}$ and no parameters, and three more are useful:

$$\begin{aligned} \mathit{twice}\,f\,x &\coloneqq f\,(f\,x), \\ \mathit{comp}\,f\,g\,x &\coloneqq f\,(g\,x), \\ \mathit{flip}\,f\,x\,y &\coloneqq f\,y\,x. \end{aligned}$$

Theorem 1.16 (two constants suffice)**.** Every definition by equations can be replaced by a combinator in K and S alone that runs the same way on every argument.

Theorem 1.19 (Turing completeness)**.** The functions on numerals that combinators in K and S compute are exactly the functions that a Turing machine computes.

Recursion is the one thing the definition seems to lack. No name may occur in its own definition, but a function may be applied to itself, and that is enough. Define

$$\mathit{self}\,f\,x \coloneqq f\,(x\,x) \qquad \text{and} \qquad \mathsf{Y}\,f \coloneqq \mathit{self}\,f\,(\mathit{self}\,f),$$

neither of which mentions itself, and run Y on any f:

$$\begin{aligned} \mathsf{Y}\,f &\rightsquigarrow \mathit{self}\,f\,(\mathit{self}\,f) && \mathsf{Y} \\ &\rightsquigarrow f\,(\mathit{self}\,f\,(\mathit{self}\,f)) && \mathit{self}, \text{ with } \mathit{self}\,f \text{ for } x. \end{aligned}$$

Write Θ for $\mathit{self}\,f(\mathit{self}\,f)$. The second line says that $\Theta \rightsquigarrow f\,\Theta$: Θ hands f a copy of itself, and that copy, when it is run, hands f another, as many times as the run asks for. A function that is to call itself takes the copy as an extra first parameter, and Y supplies it.

We've taken a lot away from our idea of a program. What's left:

- A program is a finite list of definitions and a term to evaluate.
- A definition is an equation.
- To run the term is to replace, over and over, a left-hand side by its right-hand side, until no replacement is left to make.
- We have no store, no stack, no instruction pointer. The order of steps cannot change the answer, although it may change whether we get one at all.

1.5 The Monster

Confluence promises that a term has at most one normal form, but does not promise that there is a guaranteed one. Nothing in Definition 1.1 says what may be applied to what, so a function can be applied to itself. Consider definition

$$\omega x \coloneqq x\, x$$

It is the function that applies its argument to itself. Now, apply ω to itself,

$$\omega\omega \rightsquigarrow \omega\omega \rightsquigarrow \omega\omega \rightsquigarrow \ldots$$

This is the Monster: a legal construction, the loop of Y with nothing inside it. *self* I does what ω does, and $\mathsf{Y}\,\mathsf{I}$ runs forever.

1.6 Types

Definition 1.21 (types)**.** Fix a stock of *basic types*, such as *Int*, *String* and *Bool*; write p, q, r, for them. The types are then inductively defined:

1. Every basic type p is a type.
2. If A and B are types, $A \to B$ is a type: the *function type*, the type of functions that take an A and return a B.
3. Nothing else is a type.

In one line,

$$A, B ::= p \mid (A \to B)$$

Read $A \to B$ as a contract. So, $\mathit{Int} \to \mathit{Bool}$ is the type of a test on integers, $(\mathit{Int} \to \mathit{Int}) \to \mathit{Int}$ is the type of something that takes a function, and $(p \to q) \to p$ is a type too, whatever the basic types of p and q.

Note that unlike terms which are grouped to the left, for types we group to the *right*.

Definition 1.22 (context and judgement)**.** A *context* Γ is a finite list pairing each variable in scope with a type, none listed twice:

$$\Gamma = x_1 : A_1, \ldots, x_n : A_n$$

Definition 1.23 (judgement)**.** A *judgement* is a claim

$$\Gamma \vdash_{\mathrm{KS}} M : A,$$

read "M has a type A when its variables have the types Γ gives them". When Γ is empty, it is written $\vdash_{\mathrm{KS}} M : A$.

Definition 1.24 (typing)**.** The two constants have the type schemas

$$(K) \qquad \mathsf{K} : A \to B \to A \tag{1}$$

$$(S) \qquad \mathsf{S} : (A \to B \to C) \to (A \to B) \to A \to C \tag{2}$$

where A, B and C stand for arbitrary types: each line is a *schema*, and gives one type for each choice of letters.

Let Γ be a context, M a term and A a type. A *typing* of $\Gamma \vdash_{\mathrm{KS}} M : A$ is a finite sequence of judgements

$$M_1 : A_1, M_2 : A_2, \ldots, M_n : A_n$$

whose last judgement $M_n : A_n$ is $M : A$, and in which each judgement $M_k : A_k$, read "M_k has type A_k in context Γ", is justified in one of four ways:

1. (K) M_k is K, and A_k is an instance of (1.1);
2. (S) M_k is S, and A_k is an instance of (1.2);
3. (*var*) M_k is variable x (or a ready-made constant), and $x : A_k$ is declared in Γ;
4. (*app*) there are two earlier judgements $M_i : A_i$ and $M_j : A_j$, with $i, j < k$, such that A_i is the type $A_j \to A_k$ and M_k is the term $M_i\, M_j$.

A typing whose last line is $M : A$ **witnesses** $\Gamma \vdash_{\mathrm{KS}} M : A$.

1.7 Subject Reduction

Definition 1.29 (subject reduction)**.** If $\Gamma \vdash_{\mathrm{KS}} M : A$ and $M \rightsquigarrow N$ by a K step, an S step or a ready-made step, then $\Gamma \vdash_{\mathrm{KS}} N : A$. (check course notes for proof)

Definition 1.30 (normalisation)**.** If $\Gamma \vdash_{\mathrm{KS}} M : A$, then M has a normal form.

This bans the Monster and all its kin. Ω has no normal form, so it has **no typing**; nor has any term with Ω inside it. However, some programs that do stop are refused too: $\mathsf{K}\, x\, \Omega$ has a normal form and no typing.

2 Hilbert Proofs

2.1 Formulas, Axioms and Proofs

Definition 2.1 (formulas)**.** Fix a stock of *propositional variables* $p, q, r, \ldots$, called *letters* for short. The *formulas* are defined inductively:

1. every letter is a formula;
2. if φ and ψ are formulas, so is $\varphi \to \psi$;
3. nothing else is a formula.

In one line,

$$\varphi, \psi ::= p \mid \varphi \to \psi$$

with the arrow grouping to the right, so that $\varphi \to \psi \to \chi$ is $\varphi \to (\psi \to \chi)$.

Definition 2.3 (axioms)**.** The *axioms* are all instances of the two schemas

$$\begin{array}{ll} \text{(I)} & \varphi \to \psi \to \varphi \\ \text{(II)} & (\varphi \to \psi \to \chi) \to (\varphi \to \psi) \to \varphi \to \chi \end{array}$$

where φ, ψ and χ stand for arbitrary formulas.

Definition 2.4 (proof from hypotheses)**.** Let Γ be a set of formulas, the *hypotheses*. A *proof of* φ *from* Γ is a finite sequence of formulas

$$\varphi_1, \varphi_2, \ldots, \varphi_n$$

whose last formula φ_n is φ, and in which each formula φ_k is justified in one of three ways:

1. (*axiom*) φ_k is an axiom;
2. (*hypothesis*) φ_k is a member of Γ;
3. (*modus ponens*) there are two earlier formulas φ_i and φ_j, with $i, j < k$, such that φ_i is the formula $\varphi_j \to \varphi_k$.

Definition 2.5 (provability; theorem)**.** Let Γ be a set of formulas and φ a formula. If there is a proof of φ from Γ, we write $\Gamma \vdash_{\mathrm{HJ}} \varphi$ and say that the proof *witnesses* $\Gamma \vdash_{\mathrm{HJ}} \varphi$. A formula φ with $\emptyset \vdash_{\mathrm{HJ}} \varphi$, written $\vdash_{\mathrm{HJ}} \varphi$, is a *theorem*.

Set a proof out as a box, with hypotheses Γ above the line and the formulas below it, one to a numbered line, each with its justification on the right:

	Γ	
1	φ_1	reason
2	φ_2	reason

Lemma 2.7 (identity)**.** For every formula φ, $\vdash_{\mathrm{HJ}} \varphi \to \varphi$.

Proof. The following proof, from no hypotheses, witnesses the claim for a propositional letter p. The square brackets give the formulas put for the schema's Greek letters, in the order φ, ψ, χ.

	no hypotheses	
1	$(p \to (p \to p) \to p) \to (p \to (p \to p)) \to p \to p$	(II) $[p, p \to p, p]$
2	$p \to (p \to p) \to p$	(I) $[p, p \to p]$
3	$(p \to (p \to p)) \to p \to p$	MP 1, 2
4	$p \to (p \to p)$	(I) $[p, p]$
5	$p \to p$	MP 3, 4

Lines 1, 2, 4 are matched against the two shapes, lines 3, 5 against the two lines they cite. Nothing in the five lines depends on p, so same lines with φ in place of p prove $\varphi \to \varphi$ for every formula φ. □

It turns out that **the thinking runs backwards from the goal**.

1. The goal $p \to p$ is not an axiom, so it comes by modus ponens: from some χ and $\chi \to (p \to p)$.
2. Could $\chi \to (p \to p)$ be an instance of (I)? Then χ would be p, and we would need a proof of p on its own. There cannot be one, nothing in the system proves a lone letter.
3. So try (II): Its last part, $\varphi \to \chi$, must be $p \to p$. That fixes $\varphi = \chi = p$ and leaves ψ free.
4. The two premises are then $p \to (\psi \to p)$ and $p \to \psi$. Choose $\psi \coloneqq p \to p$, and both premises are instances of (I).

2.2 The Deduction Theorem

Proposition 2.9 (stuctural facts)**.** For all sets of formulas Γ, Δ and all formulas φ, ψ:

1. (reflexivity) if $\varphi \in \Gamma$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi$;
2. (monotonicity) if $\Gamma \subseteq \Delta$ and $\Gamma \vdash_{\mathrm{HJ}} \varphi$ then $\Delta \vdash_{\mathrm{HJ}} \varphi$;
3. (transitivity) if $\Gamma \vdash_{\mathrm{HJ}} \varphi$ and $\Delta \cup \{\varphi\} \vdash_{\mathrm{HJ}} \psi$ then $\Gamma \cup \Delta \vdash_{\mathrm{HJ}} \psi$.

Proof. In each case the job is to exhibit a proof of the required kind, built from the proofs we are given; nothing is said about what the formulas mean.

1. The one-line sequence φ is a proof of φ from Γ: its only line is justified as a hypothesis, because φ is a member of Γ.
2. A proof of φ from Γ is already a proof of φ from Δ: every line justified as a member of Γ is a member of Δ too, because $\Gamma \subseteq \Delta$, and the other two kinds of line do not mention Γ.
3. Write out a proof of φ from Γ, and after it the proof of ψ from $\Delta \cup \{\varphi\}$ with every line justified as "hypothesis φ" deleted; each modus ponens that cited one of the deleted lines now cites the last line of the first proof, which is φ. Every remaining hypothesis is in Γ or in Δ, so the result is a proof of ψ from $\Gamma \cup \Delta$.

Theorem 2.12 (Deduction Theorem)**.** For every set of formulas Γ and all formulas φ and ψ,

$$\Gamma \cup \{\varphi\} \vdash_{\mathrm{HJ}} \psi \iff \Gamma \vdash_{\mathrm{HJ}} \varphi \to \psi$$

2.3 Proofs are Programs

Theorem 2.13 (Curry's correspondence)**.** Let Γ be a context and $|\Gamma|$ the set of types declared in it.

1. (Erasure) Replace every line of a typing of $\Gamma \vdash M : A$ by its partner in the right-hand column. The result is a proof of A from $|\Gamma|$.
2. (Decoration) Take a proof of φ from some hypotheses, give each hypothesis a variable, and replace every line by its partner in the left-hand column, working downwards. The result is a typing of $\Gamma \vdash M : \varphi$ for some term M.

In particular, $\Gamma \vdash_{\mathrm{KS}} M : A$ for some term M if and only if $|\Gamma| \vdash_{\mathrm{HJ}} A$

Proof. All that needs checking is that each line's justification survives the change, and the table says it does with the four cases below. Every line of a typing has the same context Γ (Definition 1.22):

1. A line by (K) has type $B \to C \to B$: erased, an instance of (I).
2. A line by (S) has type $B \to C \to D \to (B \to C) \to B \to D$: erased, an instance of (II).
3. A line by (var) has a type listed in Γ: erased, a member of $|\Gamma|$, a hypothesis.
4. A line by "(app) i, j" has type C, where line i has type $B \to C$ and line j has type B. Erased, "MP i, j" is a correct justification for C.

That is it for (1), for (2), run the same four cases upwards.

2.4 More Connectives: and, or, not

Definition 2.15 (formulas, extended)**.** . Fix a stock of letters $p, q, r, \ldots$, as before. The *formulas* are defined inductively:

1. every letter is a formula;
2. $\bot$ is a formula, read "absurd";
3. if φ, ψ are formulas, so is $\varphi \to \psi$;
4. if φ, ψ are formulas, so is $\varphi \wedge \psi$;
5. if φ, ψ are formulas, so is $\varphi \vee \psi$;
6. nothing else is a formula.

In one line,

$$\varphi, \psi ::= p \mid \bot \mid \varphi \to \psi \mid \varphi \wedge \psi \mid \varphi \vee \psi$$

Definition 2.17 (axioms, extended)**.** From here on, the *axioms* are all instances of the nine schemas

$$\begin{array}{ll}
\text{(I)} & \varphi \to \psi \to \varphi \\
\text{(II)} & (\varphi \to \psi \to \chi) \to (\varphi \to \psi) \to \varphi \to \chi \\
\text{(pair)} & \varphi \to \psi \to \varphi \wedge \psi \\
\text{(fst)} & \varphi \wedge \psi \to \varphi \\
\text{(snd)} & \varphi \wedge \psi \to \psi \\
\text{(left)} & \varphi \to \varphi \vee \psi \\
\text{(right)} & \psi \to \varphi \vee \psi \\
\text{(either)} & (\varphi \to \chi) \to (\psi \to \chi) \to \varphi \vee \psi \to \chi \\
\text{(absurd)} & \bot \to \varphi
\end{array}$$

where φ, ψ and χ stand for arbitrary formulas. A proof of φ from Γ is a box, as before, in which each line is

1. (axiom) an instance of one of the nine schemas;
2. (hypothesis) a member of Γ;
3. (modus ponens) the φ_k of earlier lines $\varphi_j \to \varphi_k$ and φ_j.

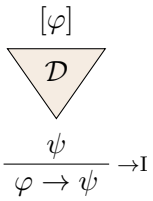
Every proof of the earlier sections is still a proof, and the proof theory of §2.2 still holds.

3 Natural Deduction

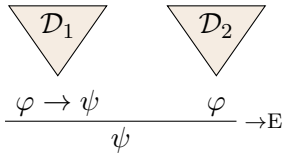
3.1 Derivations in NJ

Definition 3.1 (derivations)**.** A *derivation* is a finite tree of formulas, drawn with its root at the bottom, in which some leaves are marked as *discharged*. Its *conclusion* is the formula at its root. The derivations are defined inductively:

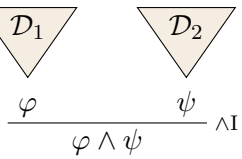
1. (hypothesis) for every formula φ, the one-node tree φ is a derivation, with conclusion φ and its leaf not discharged;
2. ($\to I$) if $\mathcal{D}$ has conclusion ψ then for every formula φ



   is a derivation.
3. ($\to E$) If $\mathcal{D}_1$ has conclusion $\varphi \to \psi$ and $\mathcal{D}_2$ has conclusion φ, then



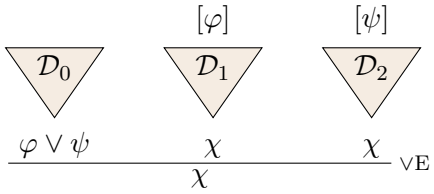
   is a derivation, with $\mathsf{OH} = \mathsf{OH}(\mathcal{D}_1) \cup \mathsf{OH}(\mathcal{D}_2)$.
4. ($\wedge I$) If $\mathcal{D}_1$ concludes φ and $\mathcal{D}_2$ concludes ψ, then



   is a derivation, with $\mathsf{OH} = \mathsf{OH}(\mathcal{D}_1) \cup \mathsf{OH}(\mathcal{D}_2)$.
5. ($\wedge E$) If $\mathcal{D}$ concludes $\varphi \wedge \psi$, then



   are derivations.
6. ($\vee I$) If $\mathcal{D}$ concludes φ, then for every formula ψ the tree with root $\varphi \vee \psi$ and immediate subtree $\mathcal{D}$ is a derivation, labelled $\vee I_1$; symmetrically from a derivation of ψ, labelled $\vee I_2$. In both cases, $\mathsf{OH} = \mathsf{OH}(\mathcal{D})$.
7. ($\vee E$) If $\mathcal{D}_0$ concludes $\varphi \vee \psi$, and $\mathcal{D}_1$ and $\mathcal{D}_2$ both conclude χ, then three trees combine into one:



8. ($\bot E$) If $\mathcal{D}$ concludes $\bot$, then for every formula χ the tree with root χ and immediate subtree $\mathcal{D}$ is a derivation.
9. Nothing else is a derivation. (Negation is an abbreviation)

Definition 3.3 (open hypotheses)**.** The *open hypotheses* of a derivation $\mathcal{D}$ are the formulas at its leaves that are not marked as discharged. We write $\mathsf{OH}(\mathcal{D})$ for the set of them.

Since derivations are defined inductively, the same set can be computed by recursion on the last rule:

$$\mathsf{OH}(\mathcal{D}) \coloneqq \begin{cases} \{\varphi\} & \text{a leaf } \varphi, \\ \mathsf{OH}(\mathcal{D}') \setminus \{\varphi\} & (\to \mathrm{I}),\ \text{discharging } \varphi, \\ \mathsf{OH}(\mathcal{D}_1) \cup \mathsf{OH}(\mathcal{D}_2) & (\to \mathrm{E}), (\wedge \mathrm{I}), \\ \mathsf{OH}(\mathcal{D}') & (\wedge \mathrm{E}), (\vee \mathrm{I}), (\bot \mathrm{E}), \\ \mathsf{OH}(\mathcal{D}_0) \cup (\mathsf{OH}(\mathcal{D}_1) \setminus \{\varphi\}) & (\vee \mathrm{E}),\ \text{discharging } \varphi, \psi, \\ \qquad \cup (\mathsf{OH}(\mathcal{D}_2) \setminus \{\psi\}) & \end{cases}$$

where $\mathcal{D}'$ is the one sub-derivation of a one-premise rule, $\mathcal{D}_1$, $\mathcal{D}_2$ the two of a two-premise rule, and $\mathcal{D}_0$, $\mathcal{D}_1$, $\mathcal{D}_2$ those of $(\vee E)$, left to right. Only two lines take something away: the $(\to I)$ and $(\vee E)$, for the two rules that discharge.

Definition 3.4 (consequence)**.** We write $\Gamma \vdash_{\mathrm{NJ}} \varphi$, and say that φ is derivable from Γ in NJ, or an NJ-consequence of Γ, when there is a derivation $\mathcal{D}$ whose conclusion is φ and whose open hypotheses are among Γ, $\mathsf{OH}(\mathcal{D}) \subseteq \Gamma$. Such a derivation *witnesses* $\Gamma \vdash_{\mathrm{NJ}} \varphi$.

3.2 Equivalence with the Hilbert System

Proposition 3.12 (derived rules)**.** For all sets of formulas Γ and all formulas φ, ψ, χ, in the Hilbert system:

1. if $\varphi \in \Gamma$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi$; (reflexivity)
2. if $\Gamma \vdash_{\mathrm{HJ}} \varphi \to \psi$ and $\Gamma \vdash_{\mathrm{HJ}} \varphi$ then $\Gamma \vdash_{\mathrm{HJ}} \psi$; (modus ponens)
3. if $\Gamma, \varphi \vdash_{\mathrm{HJ}} \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi \to \psi$; (the Deduction Theorem)
4. if $\Gamma \vdash_{\mathrm{HJ}} \varphi$ and $\Gamma \vdash_{\mathrm{HJ}} \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi \wedge \psi$; (pair)
5. if $\Gamma \vdash_{\mathrm{HJ}} \varphi \wedge \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi$; (fst)
6. if $\Gamma \vdash_{\mathrm{HJ}} \varphi \wedge \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \psi$; (snd)
7. if $\Gamma \vdash_{\mathrm{HJ}} \varphi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi \vee \psi$; (left)
8. if $\Gamma \vdash_{\mathrm{HJ}} \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi \vee \psi$; (right)
9. if $\Gamma \vdash_{\mathrm{HJ}} \varphi \vee \psi$, and $\Gamma, \varphi \vdash_{\mathrm{HJ}} \chi$, and $\Gamma, \psi \vdash_{\mathrm{HJ}} \chi$, then $\Gamma \vdash_{\mathrm{HJ}} \chi$; (either)
10. if $\Gamma \vdash_{\mathrm{HJ}} \bot$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi$; (absurd)
11. if $\Gamma \vdash_{\mathrm{HJ}} \varphi$ then $\Gamma \vdash_{\mathrm{HJ}} \psi \to \varphi$; (I)

    if $\Gamma \vdash_{\mathrm{HJ}} \varphi \to \psi \to \chi$ and $\Gamma \vdash_{\mathrm{HJ}} \varphi \to \psi$ then $\Gamma \vdash_{\mathrm{HJ}} \varphi \to \chi$. (II)

Lemma 3.13 (the axioms as trees)**.** Every instance of each of the nine schemas of HJ has a derivation in NJ with no open hypotheses.

Theorem 3.14 (equivalence with the Hilbert system)**.** For every set of formulas Γ and every formula φ built from letters and $\bot$ with $\to$, $\wedge$ and $\vee$,

$$\Gamma \vdash_{\mathrm{NJ}} \varphi \qquad \text{if and only if} \qquad \Gamma \vdash_{\mathrm{HJ}} \varphi$$

In words: the two systems, natural deduction and the Hilbert system, prove the same things from the same hypotheses.

3.3 Folding up Proofs

Theorem 3.15 (Curry–Howard)**.** Folding is a one-to-one correspondence between the derivations of NJ, with a name written on each hypothesis.
More information in course notes.

3.4 Removing Detours

Definition 3.17 (free variables)**.** The set $\mathsf{FV}(M)$ of *free variables* of a term M is defined by recursion on the shape of M:

$$\mathsf{FV}(M) \coloneqq \begin{cases} \{x\} & \text{if } M \text{ is a variable } x, \\ \mathsf{FV}(M_1) \cup \mathsf{FV}(M_2) & \text{if } M \text{ is } M_1\, M_2, \\ \mathsf{FV}(M_1) \setminus \{x\} & \text{if } M \text{ is } \lambda x.\, M_1. \end{cases}$$

A term with no free variables is *closed*.

Definition 3.18 (substitution)**.** $M[N/x]$, "M with N for x": the free occurrences of x in M replaced by N, and nothing else.

$$M[N/x] \coloneqq \begin{cases} N & \text{if } M \text{ is } x, \\ y & \text{if } M \text{ is a variable } y \neq x, \\ M_1[N/x]\, M_2[N/x] & \text{if } M \text{ is } M_1\, M_2, \\ \lambda x.\, M_1 & \text{if } M \text{ is } \lambda x.\, M_1, \\ \lambda y.\, M_1[N/x] & \text{if } M \text{ is } \lambda y.\, M_1,\ y \neq x, \text{ provided } y \notin \mathsf{FV}(N). \end{cases}$$

Definition 3.19 (β-reduction)**.** The one-step relation $M \rightsquigarrow N$, "M deduces to N", is given by the rule

$$\frac{}{(\lambda x.\, P)\, Q \ \rightsquigarrow\ P[Q/x]}\ \beta$$

together with a rule for each place a subterm can sit, which says that a step taken inside a term is a step of the term:

$$\frac{P \rightsquigarrow P'}{P\,Q \ \rightsquigarrow\ P'\,Q}\ \mathrm{app}_1 \qquad \frac{Q \rightsquigarrow Q'}{P\,Q \ \rightsquigarrow\ P\,Q'}\ \mathrm{app}_2 \qquad \frac{P \rightsquigarrow P'}{\lambda x.\, P \ \rightsquigarrow\ \lambda x.\, P'}\ \mathrm{abs}$$

The first rule is the step, called β; $(\lambda x.\, P)Q$ is the redex: the term of a detour, with the term of the straightened proof on the right.

3.5 The Simply Typed λ-Calculus

Definition 3.21 (maximal formula)**.** A *maximal formula* in a derivation is an occurrence of a formula that is the conclusion of an introduction rule and, at once, the major premise of an elimination rule. A derivation with no maximal formula is *normal*.

Definition 3.22 (normal and neutral terms)**.** The neutral and the normal terms are defined together. A term is *neutral* if it is one of:

- a variable;
- a destructor whose major part is neutral and whose other parts are normal

A term is *normal* if it is one of:

- a constructor whose parts are normal;
- a neutral term.

A neutral term is a chain of destructors hanging off a variable, it head, since nothing in the chain binds it.

Lemma 3.23 (normal terms)**.** Normal terms are the terms of normal derivations:

1. a term is normal exactly when no destructor in it has a constructor as its major part;
2. a derivation is normal exactly when the typed term it folds up to is normal.

Definition 3.24 (free variables, all connectives)**.** On the full calculus, $\mathsf{FV}(M)$ is defined by recursion on the shape of M:

$$\mathsf{FV}(M) \coloneqq \begin{cases} \{x\} & \text{if } M \text{ is a variable } x, \\ \mathsf{FV}(M_1) \cup \mathsf{FV}(M_2) & \text{if } M \text{ is } M_1\, M_2 \text{ or } (M_1, M_2), \\ \mathsf{FV}(M_1) \setminus \{x\} & \text{if } M \text{ is } \lambda x.\, M_1, \\ \mathsf{FV}(M_1) & \text{if } M \text{ is } F\, M_1, \end{cases}$$

and, for a **case**,

$$\begin{aligned} &\mathsf{FV}(\textbf{case } M_0 \textbf{ of } \{\mathit{Left}\ y \Rightarrow M_1 \mid \mathit{Right}\ z \Rightarrow M_2\}) \\ &\quad \coloneqq \mathsf{FV}(M_0) \cup (\mathsf{FV}(M_1) \setminus \{y\}) \cup (\mathsf{FV}(M_2) \setminus \{z\}). \end{aligned}$$

The new lines pass through the parts, and the **case** line removes each branch's variable from that branch, as the λ line removes the parameter from the body.

Theorem 3.29 (subject reduction)**.** If $\Gamma \vdash_\lambda M : A$ and $M \rightsquigarrow N$, then $\Gamma \vdash_\lambda N : A$
Proof in course notes.

3.6 Normalisation

Theorem 3.33 (normalisation)**.** Every typed term runs to a normal form.
Proof in course notes.

Corollary 3.35 (normal derivations)**.** Every derivation can be rewritten, by removing maximal formulas, into a normal derivation of the same formula whose open hypotheses are among those of the original.

Theorem 3.38 (consistency)**.** There is no closed term of type $\bot : \nvdash_{\mathrm{NJ}} \bot$. More generally, for every letter p, $\nvdash_{\mathrm{NJ}} p$

Corollary 3.39 (disjunction property)**.** If $\vdash_{\mathrm{NJ}} A \vee B$, then $\vdash_{\mathrm{NJ}} A$ or $\vdash_{\mathrm{NJ}} B$.

3.7 Programs as Proofs

Definition 3.44 (trasnlation into combinators)**.** Every term M built form variables by application and λ translates to a term M° built from K, S and variables by application, with no λ in it, such that:

1. M° has the same free variables as M and does the same thing to every argument: if M is $\lambda x.\, P$ then $M^\circ\, N \rightsquigarrow^* P^\circ[N/x]$ for every N;
2. M° has the types M has: if $\Gamma \vdash_\lambda M : A$ then $\lambda \vdash_{\mathrm{KS}} M^\circ : A$

Conversely every combinator term is a λ-term, by the definitions of K and S above, with the same types.